

```

1  (*
2                                     CS51 Lab 6
3      Variants, algebraic types, and pattern matching (continued)
4
5  In this lab, you will continue to use algebraic data types to create
6  several data structures. You'll create a data structure for storing
7  weather information and work with a specific type of binary tree, the
8  *binary search tree*, which allows for efficient storage and search of
9  ordered information. A particular application is the use of Gorn
10 addresses, named after the early computer pioneer Saul Gorn of
11 University of Pennsylvania, who invented the technique.
12
13 There are two parts to this lab. Please refer to the following
14 files to complete all exercises:
15
16     lab6_part1.ml -- Part 1: The Weather
17 -> lab6_part2.ml -- Part 2: Binary search trees and Gorn addresses
18 *)
19
20 (* Objective: This lab is intended to reinforce core concepts in
21    typing in OCaml, including:
22
23        Algebraic data types
24        Using algebraic data types to enforce invariants
25        Implementing polymorphic algebraic data types
26    *)
27
28 (*=====
29     Part 2: Binary search trees and Gorn addresses
30
31 Recall from Chapter 11 of the textbook that binary trees are a data
32 structure composed of nodes that store a value from some base type as
33 well as a left and a right subtree. To well-found this recursive
34 definition, a binary tree can also be empty. Defined in this way, binary
35 trees resemble lists, but with two "tails".
36
37 We'll use the definition of polymorphic binary trees from the
38 textbook, reproduced and annotated here to clarify the arguments of
39 the `Node` value constructor:
40 *)
41
42 type 'a bintree =
43   | Empty
44   | Node of 'a * 'a bintree * 'a bintree ;;
45   (*
46      ^           ^           ^
47      |           |           |
48      value      left      right
49      at node    subtree   subtree
50   *)
51
52 (*.....
53 Exercise 8: Define a function `node_count : 'a bintree -> int`, which
54 returns the number of internal nodes (that is, not including the empty

```

```

53 trees) in a binary tree.
54 .....*)
55
56 let node_count =
57   fun _ -> failwith "node_count not implemented" ;;
58
59 (* A *binary search tree* is a binary tree that obeys the following
60 invariant:
61
62 .....
63 For each node in a binary search tree, all values stored in its
64 left subtree are less than the value stored at the node, and all
65 values stored in its right subtree are greater than the value
66 stored at the node.
67 .....
68
69 (For our purposes, we'll take "less than" to correspond to OCaml's
70 polymorphic `<` operator.)
71
72 For example, the following integer binary tree is a binary search
73 tree:
74 *)
75
76 let example1 =
77   Node (10, Node (5, Empty,
78                   Node (7, Empty,
79                         Node (9, Empty, Empty))),
80         Node (15, Empty, Empty))
81
82 (* This tree can be depicted graphically (given the limitations of
83 ascii art) as
84
85      10
86      ^
87     /  \
88    5    15
89    ^    ^
90   /      \
91  7          \
92  ^            \
93  \              \
94  9                \
95  ^                  \
96
97 The binary tree in Figure 11.3(b) in the textbook, duplicated here as
98 the `string bintree` `example2`, also happens to be a binary search
99 tree. Do you see why it obeys the invariant? *)
100
101 let example2 =
102   Node ("red",
103         Node ("orange",
104               Node ("green", Node ("blue", Empty, Empty),
105                                   Node ("indigo", Empty, Empty)),
106               Empty),

```

```

107         Node ("yellow", Node ("violet", Empty, Empty),
108                     Empty)) ;;
109
110 (* Binary search trees are useful because, as indicated by the name,
111    searching for a value in a binary search tree is especially efficient.
112    Rather than needing to search for a value throughout the whole tree,
113    the value stored at a node tells you determinately whether to search
114    in the left or the right subtree. Other functionality, like finding
115    the minimum or maximum value, are also especially efficient in binary
116    search trees. *)
117
118 (*.....
119 Exercise 9: Define a function `find_bst` for binary search trees, such
120 that `find_bst tree value` returns `true` if `value` is stored at some
121 node in `tree`, and `false` otherwise. For instance,
122
123     # find_bst example1 9 ;;
124     - : bool = true
125     # find_bst example1 10 ;;
126     - : bool = true
127     # find_bst example1 100 ;;
128     - : bool = false
129     # find_bst example2 "yellow" ;;
130     - : bool = true
131     .....*)
132
133 let find_bst = fun _ -> failwith "find_bst not implemented" ;;
134
135 (*.....
136 Exercise 10: Define a function `min_bst`, such that `min_bst tree`
137 returns the minimum value stored in binary search tree `tree` as an
138 option type, and `None` if the tree has no stored values. For
139 instance,
140
141     # min_bst example1 ;;
142     - : int option = Some 5
143     # min_bst example2 ;;
144     - : string option = Some "blue"
145     # min_bst Empty ;;
146     - : 'a option = None
147     .....*)
148
149 let min_bst (tree : 'a bintree): 'a option =
150     failwith "min_bst not implemented" ;;
151
152 (* Constructing binary search trees must be done carefully so that the
153    invariant is always preserved. Next, you'll implement a function for
154    adding a value to a binary search tree, while maintaining the
155    invariant. *)
156
157 (*.....
158 Exercise 11: Define a function `insert_bst : 'a -> 'a bintree -> 'a
159 bintree` such that if `tree` is a binary search tree, `insert_bst
160 value tree` returns a tree with the same elements as `tree` but also

```

```

161 with the new `value` inserted. (If the value is already in the tree,
162 the tree can be returned unchanged.) Make sure that the tree that is
163 returned maintains the binary search tree invariant.
164
165 For instance, your function should have the following behavior.
166
167     # let example1_again =
168         Empty
169         |> insert_bst 10
170         |> insert_bst 5
171         |> insert_bst 15
172         |> insert_bst 7
173         |> insert_bst 9 ;;
174     val example1_again : int bintree =
175         Node (10, Node (5, Empty, Node (7, Empty, Node (9, Empty, Empty))),
176             Node (15, Empty, Empty))
177
178 (The returned tree is the same one as `example1` depicted above.)
179 .....*)
180
181 let rec insert_bst (value : 'a) (tree : 'a bintree) : 'a bintree =
182     failwith "insert_bst not implemented" ;;
183
184 (* The *Gorn address* of a node in a tree (named after the early
185 computer pioneer Saul Gorn of University of Pennsylvania
186 <https://en.wikipedia.org/wiki/Saul\_Gorn>, who invented the technique)
187 is a description of the path to take from the root of the tree to the
188 node in question. For a binary tree, the elements of the path specify
189 whether to go left or right at each node starting from the root of the
190 tree. We'll define an enumerated type for the purpose of recording the
191 left/right moves. *)
192
193 type direction = Left | Right ;;
194
195 (* Thus, for the tree `example1` defined above, the Gorn address of the
196 root is `[]` and the Gorn address of the node containing the value `9`
197 is `[Left, Right, Right]`. *)
198
199 (* .....
200 Exercise 12: Define a function `gorn : 'a -> 'a bintree -> direction
201 list` that given a target value and a binary search tree returns the
202 Gorn address of the target value in the tree. It should raise a
203 `Failure` exception if the value doesn't occur in the tree. For
204 instance,
205
206     # gorn 9 example1 ;;
207     - : direction list = [Left; Right; Right]
208     # gorn 10 example1 ;;
209     - : direction list = []
210     # gorn 100 example1 ;;
211     Exception: Failure "gorn: value not found".
212 .....*)
213
214 let rec gorn (target : 'a) (tree : 'a bintree) : direction list =

```

```
215     failwith "gorn not implemented" ;;
```